House Recommendation System

How the code works — with a focus on the Machine Learning (KNN)

---

1. The Big Picture

The app recommends houses in Bangladesh from your preferences. The code is split so each file has one clear job:

```
CSV file ■■> load_data.py ■■> houses.db (SQLite)
                                  |
       app.py (UI) ■■imports■■> recommender.py ■■reads■■> houses.db
            |
            |■■> nlp_search.py   (OpenAI: sentence -> filters)
            |■■> auth.py         (favorites / history in users.db)
```

When you run `streamlit run app.py`:

1. **ensure_database()** (load_data.py) builds houses.db from the CSV if it does not exist yet. Runs once.
2. **auth.init_user_db()** creates the favorites / history tables.
3. The OpenAI key is bridged from secrets.toml into the environment.
4. **main_app()** draws the sidebar and sections and waits for your click.

2. Each File, One Line Each

File	Job
load_data.py	CSV -> cleaned SQLite. The data-prep step.
recommender.py	The ML brain (scoring + KNN). The focus below.
nlp_search.py	Sends your sentence to OpenAI, gets structured filters back.
auth.py	Saves favorites and search history.
app.py	The Streamlit UI that glues everything together.
main.py	Optional REST API (not used by the Streamlit app).

3. The Machine Learning

There are **two ML pieces**, both built on the same foundation: **learned feature scaling**.

Step A — Turn each house into numbers (a feature vector)

A house mixes numbers (price, area, bedrooms, bathrooms) with text (size category). ML needs everything as numbers on a comparable scale:

```
NUMERIC_FEATURES = ["Budget_BDT", "Area_sqft", "Bedrooms", "Bathrooms"]
SIZE_ORDINAL = {"small": 0.0, "medium": 0.5, "large": 1.0}
```

The scaling problem: budget is in millions (9,000,000) while bedrooms is a single digit (3). Compared raw, budget would drown out bedrooms. So we normalise every feature to 0–1 with

scikit-learn's **MinMaxScaler**, which *learns* the real min and max from the data:

```
scaled_value = (value - min) / (max - min)
```

Worked example — bedrooms range from 1 to 5 in the data:

- a 1-bed -> (1-1)/(5-1) = **0.0**
- a 3-bed -> (3-1)/(5-1) = **0.5**
- a 5-bed -> (5-1)/(5-1) = **1.0**

This is done once and cached:

```
@lru_cache(maxsize=1)
def _fit_model():
    df = load_houses()
    num = _numeric_frame(df)                # the 4 numeric columns
    scaler = MinMaxScaler().fit(num)        # LEARN min/max from data
    scaled = scaler.transform(num)          # every house -> 0..1
    size = ...map(SIZE_ORDINAL)...          # size category -> 0/0.5/1
    feats = np.hstack([scaled, size])       # 5-number vector per house
    nn = NearestNeighbors(metric='cosine').fit(feats)   # the KNN model
    return df, scaler, feats, nn
```

So **every house becomes a 5-number vector** [budget, area, bedrooms, bathrooms, size], each between 0 and 1.

Step B — recommend(): score houses against YOUR preferences

Your search is put into the same 5-number space. For each feature you gave, it computes a similarity from 0 to 1:

```
similarity = 1 - |your_scaled_value - house_scaled_value|
```

Identical -> 1.0 (100%). Far apart -> near 0. Features are then combined by **weight** (budget matters more than bathrooms):

```
DEFAULT_WEIGHTS = {"budget":35, "area":20, "bedrooms":15,
                   "bathrooms":10, "house_size":10, "location":10}

score = (sum of weight x similarity) / (sum of weights) x 100
```

That per-feature similarity is exactly what fills the **“Why this match?”** bars in the UI. Before scoring, **hard filters** drop anything outside your district, over budget, or over the price-per-sqft limit.

4. KNN — how “Similar houses” works

This is the classic **k-Nearest-Neighbours** algorithm, used in `similar_to()`.

The idea of KNN

Picture every house as a **dot in space** — not a 2D map, but 5D (one axis per feature). Houses that are alike (similar price, size, beds…) sit **close together**; very different houses sit far apart. “Find the k nearest neighbours” literally means **find the k closest dots** to a given house. No training, no learned weights — KNN just measures distance and returns the closest points.

How it is wired in this project

At startup the model was already built on all 7,715 house vectors:

```
nn = NearestNeighbors(metric="cosine").fit(feats)   # feats = all house vectors
```

When you open “Similar houses” on a saved favorite, similar_to(house) runs:

```
def similar_to(house, top_n=5):
    df, scaler, feats, nn = _fit_model()

    # 1. Put THIS house into the same 5-number space
    raw = pd.DataFrame([[house[c] for c in NUMERIC_FEATURES]], ...)
    scaled = scaler.transform(raw)               # same scaler as everyone
    size = SIZE_ORDINAL.get(house['House_Size'].lower(), 0.5)
    vec = np.hstack([scaled, [[size]]])          # the query point

    # 2. Ask the model: which houses are closest?
    distances, indices = nn.kneighbors(vec, n_neighbors=k)

    # 3. Return them, distance -> match %
    for dist, i in zip(distances[0], indices[0]):
        row = df.iloc[i].to_dict()
        if same_as_query(row): continue          # skip the house itself
        row['match_score'] = (1 - dist) * 100     # closer = higher %
```

Three steps: **(1) Encode** the chosen house into the 5-number vector; **(2) nn.kneighbors(...)** returns the k closest houses (their distances and row indices); **(3) convert distance to a match %** and show them (skipping the house itself, which is always its own nearest neighbour at distance 0).

What “distance” means here — cosine

We used `metric="cosine"`. **Cosine distance** compares the shape / direction of two vectors:

- cosine similarity 1 -> identical profile -> distance 0 -> **100% match**
- cosine similarity 0 -> very different -> distance 1 -> **0% match**

So `match_score = (1 - distance) x 100`. A 3-bed medium flat at a similar price/area comes out near the top; a 5-bed luxury villa is far away.

Why KNN fits this project

- **No training data needed** — there is no “users who bought X also bought Y” data yet, so collaborative ML isn't possible. KNN only needs the house features you already have.
- **Explainable** — “these are the closest houses in feature space” is easy to justify.
- **Instant** — the model is fit once and cached; each lookup is a fast distance query.

5. Quick Summary

Piece	Technique	Where	Job
Feature scaling	MinMaxScaler	_fit_model()	Make features comparable (0-1)
Preference ranking	Weighted similarity	recommend()	Rank houses vs. your filters + explain
Similar houses	KNN (cosine)	similar_to()	Find k closest houses to one you liked

One-sentence version: every house becomes a 5-number fingerprint; recommend() scores those fingerprints against your wishes, and KNN finds the fingerprints nearest to a house you already liked.

6. Beyond KNN — Could another model do better?

A natural question: is k-Nearest-Neighbours the best choice, or would a different model give better recommendations? The honest answer depends on one thing — **what data we have**.

This project has **house features only** (price, area, bedrooms, bathrooms, size). There is **no logged user behaviour** yet — no record of who viewed, clicked, or favourited which house. That single fact rules out the models people usually reach for:

The landscape of options

Model / change	Better than KNN here?	Why
KNN + weighted Euclidean (what we did)	Yes - cheap, clear win	Fixes the 'everything looks ~98%' problem. Ranks by real closeness in price/size,
Clustering (KMeans / HDBSCAN)	Sideways	Groups houses into segments - useful for 'show me this type', but coarser than KN
Collaborative filtering (SVD, ALS) / neural two-tower	Not possible yet	Needs user-item interaction history to learn from. We only have house features +
Learning-to-rank (LightGBM ranker)	Best - but needs data first	Learns the true ranking from real favourites/clicks. The genuine upgrade path onc
Deep embeddings / neural net	Overkill	~7,700 rows and ~5 features - a neural net won't beat weighted KNN and adds a l

Bottom line: the fancier models (collaborative filtering, neural recommenders) need user-behaviour data we don't collect yet. With the data we have, the best move is not to replace KNN but to **make it smarter** - which is exactly the upgrade below.

7. The upgrade: cosine -> weighted Euclidean KNN

The original KNN used `metric="cosine"`. Cosine distance compares only the *direction* of the feature vectors, not their magnitude. In practice that made almost every neighbour score ~98%, even for houses that were far apart in real price and size.

The problem, seen on a real seed

Seed = a rare 10-bed, 9,000 sqft, 50,000,000 BDT house. Under cosine, the 2nd 'most similar' house was a 6,357 sqft / 29,000,000 BDT home - 21 million BDT cheaper - yet it still scored ~98%. Cosine simply couldn't tell those apart.

What changed

Two edits in `recommender.py`:

1. Weighted features. Each feature is multiplied by the square root of its normalised importance (the same idea as DEFAULT_WEIGHTS), so budget and area count more than a bathroom. Multiplying by sqrt(weight) turns plain Euclidean distance into a *weighted* Euclidean distance.

```
KNN_FEATURE_WEIGHTS = {"budget":35, "area":20, "bedrooms":15,
                       "bathrooms":10, "house_size":10}

def _knn_weight_vector():
    w = np.array([...], float)      # in feature-column order
    return np.sqrt(w / w.sum())     # sqrt of normalised weights

feats = np.hstack([scaled, size]) * _knn_weight_vector()
nn = NearestNeighbors(metric="euclidean").fit(feats)
```

2. Euclidean distance + a proper score. Euclidean measures true closeness in the 5-number space. Distance is turned into a match % with `100 / (1 + distance)`: distance 0 -> 100%, and the score decays smoothly as houses get further apart.

```
# weighted-Euclidean distance -> similarity %
row["match_score"] = round(100.0 / (1.0 + float(dist)), 1)
```

Before vs. after (same luxury seed)

Neighbour (7bd/8ba)	Cosine score	Weighted-Euclidean score
7,560 sqft / 60M BDT	~98.4%	84.4% (now ranked #1)
6,880 sqft / 60M BDT	~98.3%	83.3%
6,500 sqft / 40M BDT	~98.8%	82.5%
6,357 sqft / 29M BDT	~98.5% (ranked #2)	80.1% (now ranked last)

The scores now **spread out meaningfully** (84% down to 80% instead of a flat 98%), and the house closest in absolute size/price is correctly ranked first. Identical houses still score a true 100%. All existing unit tests still pass.

What to do next (the real next level)

Start **logging user feedback** - which recommended houses people favourite or click. Once that history exists, a **LightGBM learning-to-rank** model can learn the actual ranking users prefer, instead of assuming feature-distance equals relevance. That is the one model that genuinely beats KNN for this task - but only after the data is there.